

RESTful Schema-Aware Data Integration API An

Object-Oriented Approach to Decoupling Clients
and Databases **Dijith Dinesh**

Department of Computer Science and Engineering
Model Engineering College, Thrikkakara
July 14, 2025

Table of Contents

Introduction

Problem Statement

Proposed Solution

System Architecture

Key Features & OOP

Conclusion

The Core Idea: A Universal API Layer

- ▶ A RESTful API server, built in Java, that acts as a smart middleman between client applications and a database.
- ▶ **Main Goal:** To completely *decouple* clients from the backend database schema.
- ▶ Empowers clients to request data exactly how they need it, without requiring any backend code changes.
- ▶ Heavily leverages Object-Oriented principles for a robust, flexible, and extensible design.

The Brittleness of Tight Coupling

The Central Problem

Client applications are often **tightly coupled** to the database schema.

- ▶ A simple backend change (e.g., renaming a column) becomes a **"breaking change"** for all clients.
- ▶ This leads to:
 - ▶ Costly and repetitive update cycles.
 - ▶ Repetitive boilerplate code on the client-side.
 - ▶ Significant security risks by indirectly exposing database structure.
- ▶ **Real-world consequences:**
 - ▶ Mars Climate Orbiter failure (unit mismatch).
 - ▶ Knight Capital's \$440M loss (deployment bug).

Our Solution: A Dynamic & Abstracted API

Our solution is a standalone API server with three pillars of functionality:

Dynamic Querying

Clients filter, sort, and paginate using simple URL parameters. No SQL needed.

On-the-Fly Transformation

Request data in JSON, CSV, XML, etc., and rename fields dynamically.

Secure Abstraction

The database schema is hidden. Access is controlled via role-based API keys.

Example API Call

GET

/data/products?format=csv&where=price:gt:100&replace=prod_name:n

System Architecture

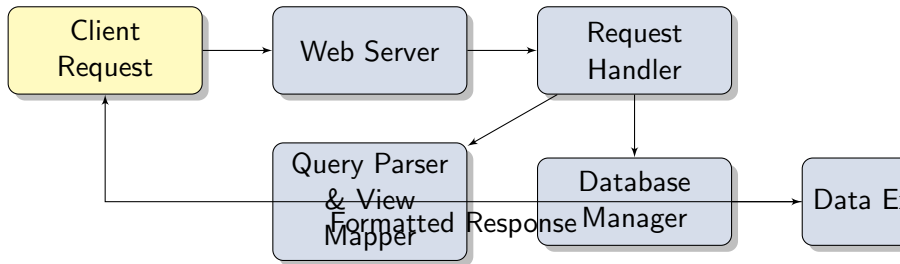


Figure: High-level data flow from request to response.

Object-Oriented Programming in Action

▶ **Abstraction:**

- ▶ The entire API is an abstraction layer.
- ▶ 'DatabaseManager' hides all raw JDBC/SQL complexities.

▶ **Encapsulation:**

- ▶ 'DataRequestHandler' encapsulates logic for the '/data' endpoint.
- ▶ Components like 'ApiQueryParser' hide their complex internal state.

▶ **Inheritance:**

- ▶ 'BaseRequestHandler' provides common functionality to all handlers.
- ▶ 'BaseExporter' provides a common structure for all format exporters (JSON, CSV, etc.).

▶ **Polymorphism:**

- ▶ The system uses any 'Exporter' object ('JsonExporter',

Conclusion & Key Takeaways

Achievements

- ▶ Successfully designed a functional, schema-agnostic RESTful API server.
- ▶ Solved the problem of tight coupling, making development more independent and robust.
- ▶ Empowered clients with dynamic querying, formatting, and field mapping.
- ▶ Demonstrated the power of OOP to build a modular and extensible system.

Future Scope

Thank You

Questions?